

Ch 05: Pointers and Arrays

The C Programming Language (2^{ed})

Derek Harter

Department of Computer Science
East Texas A&M University

Summer 2026



EAST TEXAS A&M

Pointers and Addresses

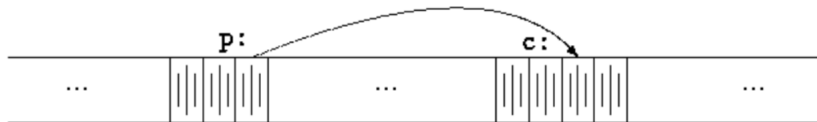


Figure 1: A pointer holds an address that points to another variable (Kernighan & Ritchie, [1988](#), pg.83)

```
int c;  
int *p;  
p = &c;
```

Pointer Variable in C

A pointer is a variable that contains the **address** of another variable.

- The unary operator & gives the **address** of an object
- The unary operator * is the **indirection** or **dereferencing** operator
 - it *follows* the pointer and dereferences what it points to
- Pointer variables have a type
 - When you dereference a pointer, the bits are treated as the declared type



EAST TEXAS A&M

Pointers, Types and Operators

- A pointer is constrained to point to a particular kind of object
 - every pointer points to a specific data type (or block thereof)
 - Exception: (void *) generic pointer but can't be dereferenced
- If ip points to the integer x, then *ip can be used anywhere x can.
 - The unary operators * and & bind more tightly than arithmetic operators

```
int x, y, *ip;  
ip = &x;
```

```
y = *ip + 1;  
*ip = *ip + 10;  
*ip += 1;
```

```
++*ip; // works as expected, preincrement x  
(*ip)++; // careful, unary operators  
// associate right to left?! more later
```

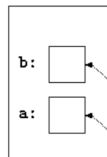


Pointers and Function Arguments

- C passes argument to functions by value (a copy is made)
 - Our version of C does not support newer “reference” parameters
- Can use pointers to effectively pass by reference

```
void  
swap(int *px, int *py)  
{  
    int temp;  
  
    temp = *px;  
    *px = *py;  
    *py = temp;  
}
```

in caller:



in swap:

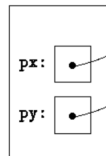


Figure 2: Pass pointer references to allow function to effect parameters of caller (Kernighan & Ritchie, 1988, pg.86)



Pointers and Arrays (1 of 2)

- In C there is a strong relationship between pointers and arrays
- An array declaration defines a block of 10 consecutive objects
 - named `a[0]`, `a[1]`, ..., `a[9]`

```
int a[10]; // a block of 10 ints
```

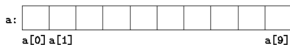


Figure 3: An array of 10 integers (Kernighan & Ritchie, 1988, pg.87)

- The notation `a[i]` refers to the *i*-th element of array `a`.
- If `pa` is a pointer to an integer, then we can point it to the 0-th element like this:

```
int *pa;  
pa = &a[0];
```

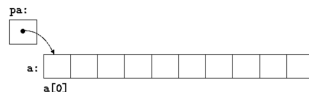


Figure 4: `pa` points to `a[0]` (Kernighan & Ritchie, 1988, pg.87)



Pointers and Arrays (2 of 2)

- if `pa` points to a particular element of an array, then `pa+1` point to the next element.
 - This offset is based on type, e.g. a pointer to an int actually increases address by 4
- `*(pa+1)` refers to `a[1]`, `*(pa+2)` refers to `a[2]`, `*(pa+i)` refers to `a[i]`

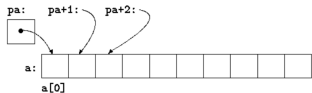


Figure 5: If `pa` points to `a[0]`, then `pa+i` indexes the i -th element of `a` (Kernighan & Ritchie, 1988, pg.88)

Pointer Indexing

The meaning of “adding 1 to a pointer”, and by extension all pointer arithmetic, is that `pa+1` points to the next object, and `pa+i` points to the i -th object beyond `pa`. This is regardless of type or size of variable pointed at.



Passing Arrays to Functions

- When an array name is passed to a function, what is passed is the location of the initial element (pointer to element 0)
- In C “strings” are arrays of characters, terminated by the null `'\0'` character (more in a bit on C string character arrays)
- Thus a canonical `strlen()` implementation takes a pointer to an array of characters, and would look like the following
 - See `kernel/string.c` for Xv6 string library implementations

```
// mystrlen: return length of string s
int
mystrlen(char *s)
{
    int n;

    for (n = 0; *s != '\0'; s++)
        n++;

    return n;
}
```



Address Arithmetic



EAST TEXAS A&M

Character Pointers and Functions (1 of 3)

- In C, a **string constant** written as “I am a string” is represented somewhere in memory as an array of characters
 - the array is terminated with a null character `\0`, (byte pattern 00000000)
 - length in storage is thus one more than the number of characters between double quotes

- Consider these declarations, similar but subtle and important difference

// an array, allocated on stack

```
char amessage[] = "now is the time";
```

// a pointer, string in constant sect

```
char *pmessage = "now is the time";
```

- `amessage` is allocated an array on function call stack, just big enough to hold 15 characters plus one more, the null character.
 - can change individual elements
- `pmessage` is a pointer to a character array allocated in global constants
 - cannot modify a constant string



EAST TEXAS A&M

Character Pointers and Functions (2 of 3)

- If you copy a string pointer like `s = t`, this only copies the pointer, both refer to same memory.
- To copy characters need a loop
- Version 1 (again see `kernel/string.c` `strncpy()` and `safestrcpy()` for xv6 library implementation)

// strcpy (version 1): copy t to s; array subscript version

```
void
strcpy_v1(char *s, char *t)
{
    int i;

    i = 0;
    while ((s[i] = t[i]) != '\0')
        i++;
}
```



Character Pointers and Functions (3 of 3)

For contrast, here is a second version using pointers and pointer arithmetic

```
// strcpy (version 2): copy t to s  
// pointer version  
void  
strcpy_v2(char *s, char *t)  
{  
    while ((*s = *t) != '\0') {  
        s++;  
        t++;  
    }  
}
```

Canonical C version you might see in our code

```
// strcpy (version 4): copy t to s  
// pointer version 3  
void  
strcpy_v4(char *s, char *t)  
{  
    while ((*s++ = *t++))  
        ; // do nothing  
}
```



Arrays of Pointers

- Since pointers are variables, they can be stored in arrays just as other variables
 - `char *argv[]` is an array of pointers, more in a bit
- Notice initialization of array of pointer to constant strings here

```
int i;
char *wordlist[] = {"charlie", "bravo", "delta", "alfa"};

// array of 4 (char *) pointers
printf("sizeof(wordlist): %ld\n", sizeof(wordlist));

// can access "string" indexed into wordlist
for (i = 0; i < 4; i++) {
    printf("wordlist[%d] = <%s>\n", i, wordlist[i]);
}
```



Command Line Arguments

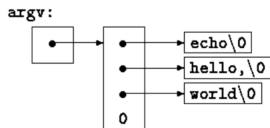


Figure 6: Command line `argv` array of `char *` pointers (Kernighan & Ritchie, 1988, pg.103)

- When `main()` is called, it is passed in any command line arguments
 - `int argc` the argument count
 - `char *argv[]` an array of pointers, one for each command line argument
- `argv[0]` always contains the name by which program was invoked

```
// echo command line arguments
```

```
int
```

```
main(int argc, char *argv[])
```

```
{
```

```
    int i;
```

```
    for (i = 0; i < argc; i++)
```

```
        printf("argv[%d]: <%s>\n",  
               i, argv[i]);
```

```
}
```

```
$ ex07 hello world
```

```
argv[0]: <ex07>
```

```
argv[1]: <hello>
```

```
argv[2]: <world>
```



EAST TEXAS A&M

Summary

- point 1
- point 2



Kernighan, B. W., & Ritchie, D. M. (1988). *The c programming language* (2nd). Pearson.

